
From two tools to one system

A context layer, a strategist, and a measurement loop that actually closes — turning content generation and GEO tracking into a single product.

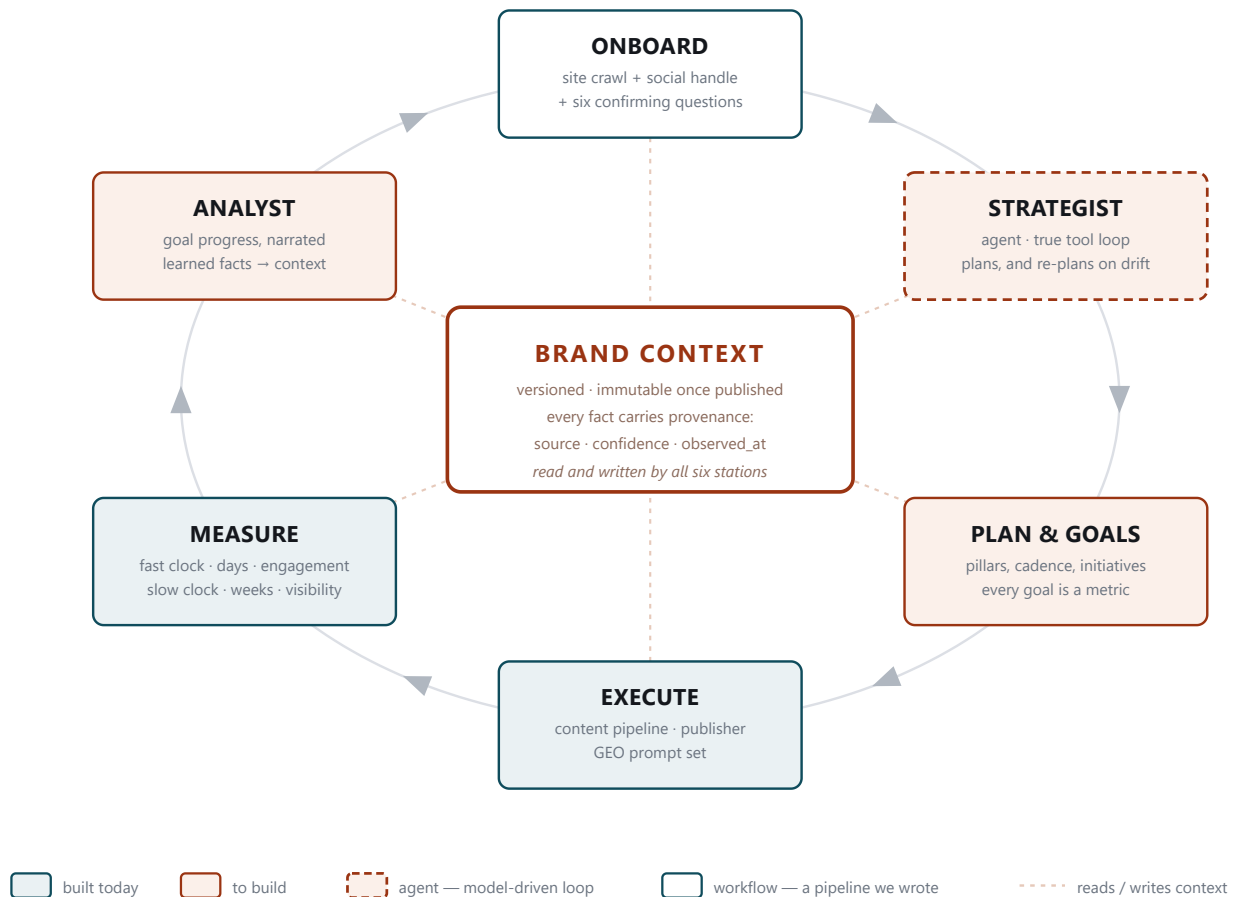
Today we have two good tools that don't know about each other.
Content generation makes assets. GEO measures AI-answer visibility. A customer using both gets no more than the sum of the parts, because nothing carries an understanding of the business between them.

The product is the loop, not the agents.

Date	Status	Extends	Horizon
2 August 2026	Proposal — for team review	docs/architecture.md	~12 weeks, two engineers

The loop

Six stations around one hub. Every station reads the Brand Context and writes back to it; that shared record is what makes the parts add up to a system rather than a bundle.



The loop. Flow runs clockwise. Only the Strategist is a true agent; everything else is a pipeline with LLM steps in it — a distinction that governs cost, latency and failure modes, and one we should hold to in review.

Five corrections to the original sketch

The shape — onboard, plan, execute, measure, re-plan — is right, and it maps cleanly onto what already exists. Five things are worth settling before a line of it gets written.

1 Most of these are workflows, not agents

An *agent* is a model-driven loop that chooses its own next step. A *workflow* is a pipeline we wrote, with LLM calls at some steps. They cost, fail and scale very differently.

Almost everything we described is a workflow — and both existing workers are already correctly built that way (`brief → image → QA → copy` , `probe → analyse → roll-up`). That is a strength. Do not convert them.

COMPONENT	ACTUALLY IS	WHY
Onboarding	WORKFLOW	Fixed steps, known output shape
Strategist	AGENT	Must explore: read site, GEO data, competitors, then decide
Researcher (later)	AGENT	Open-ended: why do answers cite these sources and not us?
Content pipeline	WORKFLOW built	Deterministic, cost-sensitive, high volume
Publisher	WORKFLOW	It is an API call with a retry policy
GEO pipeline	WORKFLOW built	It is a measurement instrument — non-determinism is a bug
Analyst	WORKFLOW + one narration call	Metrics are computed, not reasoned
Brand critic	single LLM-as-judge call	One decision, no loop

THE CALL

Two agents, six workflows. If we call all eight "agents" we will spend a month building loop infrastructure we don't need, and our unit costs go non-deterministic. Multi-agent is a good *product story* and a poor *implementation default*.

2 The measurement loop has two clocks — don't merge them

"GEO tracks analytics after each campaign" will not survive scrutiny. GEO measures what answer engines say about a brand. That moves over weeks to months and is driven by web presence, citations, listings and reviews — **not by an Instagram carousel posted on Tuesday**. If we show a customer "we posted, your GEO score rose," we are claiming causality we cannot support, and the first person to check will find the confound.

LOOP	PERIOD	MEASURES	SOURCE	STEERS
Fast	days	reach, engagement, saves, clicks	Instagram Graph API	content tactics
Slow	2–4 weeks	presence, position, share of voice, citations	GEO probes (built)	strategy & positioning

GEO's real output isn't "post more." It is *"you are absent from six of nine discovery prompts; the answers that beat you cite four sources you appear on none of."* That drives a strategy revision — get listed, get reviewed, publish comparison content — which then drives content work. The causal chain runs **GEO → strategy → content**, not content → GEO.

ATTRIBUTION RULE TO ADOPT NOW

Never claim a GEO movement was caused by a campaign. Put both series on one timeline, annotate campaigns as markers, and let the correlation be visible without asserting it. More honest — and, in a demo, more credible.

3 The questionnaire should be a confirmation, not an interrogation

Small-business owners abandon long forms. We already have a better input: the content branch ships `core.brand_site_profiles` — website crawl and extraction.

Onboarding should be **URL + Instagram handle → we derive the profile → the owner confirms or corrects six things**. The questionnaire's job is only what cannot be inferred: goals, effort ceiling, geography, things we must never say, and what success means to them. Everything else we read. Better product, and half of it exists.

4 A goal that isn't a metric is decoration

If the Strategist can emit "increase brand awareness," the plan is unfalsifiable and the whole system is theatre. Every goal must compile to a series we already compute — or the planner rewrites it until it does.

```
goal    := { metric, target, baseline, horizon, scope }

metric ∈ { geo_score, presence_rate, share_of_voice, average_position,
           citation_rate, posts_published, engagement_rate, reach }
```

The planner's output is validated against this enum **by code, not by prompt**. Anything that doesn't compile is rejected and retried. This single constraint is what makes the plan reviewable, the progress chart real, and automatic re-planning possible at all.

5 Build the context layer as a record and a function, not a platform

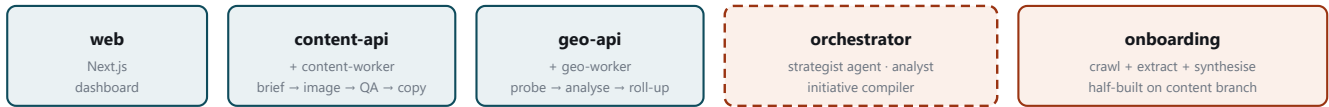
"Context layer" invites building infrastructure first. Resist. It is one versioned table, `core.brand_context`, and one function, `assembleContext(brandId, task)`, returning a typed, token-budgeted bundle for a given consumer. A few hundred lines.

Vector search, embeddings and a retrieval service are a Phase 4 conversation, and only if the corpus outgrows a prompt window. Ship the record first — every agent reading the same record is ninety per cent of the value.

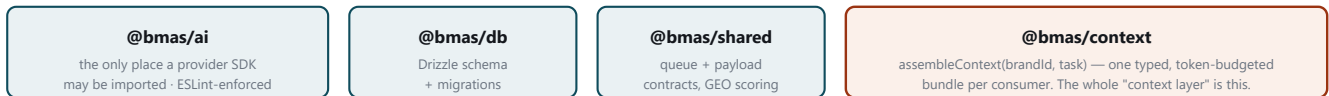
What we actually build

Three additions to a codebase that already has the hard parts: a context record, a plan schema, and an event log. No new orchestration framework.

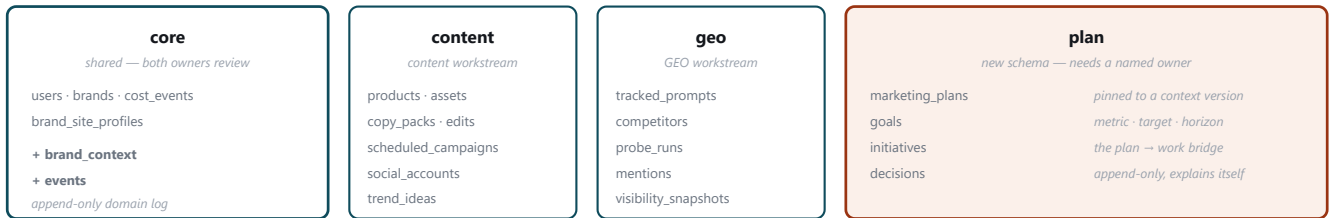
APPS



PACKAGES — SHARED, BOTH OWNERS REVIEW



POSTGRES — ONE DATABASE, SCHEMA-NAMESPACED BY OWNER



System map. Teal is built and running today; terracotta is new. Keeping `plan` out of `core` stops it becoming a third contested surface between the two workstreams.

Brand Context — the spine

One row per brand per version, immutable once published. Agents pin the version they used, so we can always answer *"what did the system believe when it decided this?"*

```
core.brand_context
  id, brand_id, version, status(draft|active|superseded)
  identity      jsonb   name, tagline, category, story
  offerings     jsonb   products/services, price band, hero items
  audience      jsonb   segments, geography, language
  positioning   jsonb   differentiators, proof points, competitive set
  constraints   jsonb   never-say list, claims we can't make, compliance
  voice         jsonb   tone, vocabulary, banned phrases
  facts         jsonb[]  { key, value, source, confidence, observed_at }
```

`facts[]` is the important part. Every assertion carries **provenance**:

SOURCE	EXAMPLE FACT
onboarding	Owner said the goal is weekend footfall
site_crawl	Menu lists 40 items across 3 locations
social	Posts twice weekly; best engagement on food reels
geo_observed	Answer engines describe the brand as "budget-friendly"
human_edit	Owner corrected the category

Provenance is what lets the context **update itself from measurement** — the outer arrow in the loop — and what lets us show a customer *why* we believe something. The `geo_observed` facts are especially valuable: how the internet's AI describes you is often not how you describe yourself, and that gap is a genuine, sellable insight.

Plan — where strategy becomes work

plan.marketing_plans	brand_id, version, context_version , status, positioning, pillars[], channel_mix, cadence, geo_themes[], rationale, created_by_agent, model
plan.goals	plan_id, metric, target, baseline, horizon, scope, status(on_track at_risk met missed), current_value
plan.initiatives	plan_id, goal_ids[], kind(content geo listing), spec jsonb, cadence, status
plan.decisions	brand_id, at, actor, decision, evidence jsonb

`plan.initiatives` is the bridge — the only place a plan becomes work. A `content` initiative compiles to generation jobs; a `geo` initiative compiles to rows in `geo.tracked_prompts`.

THE MOMENT THE TWO PRODUCTS BECOME ONE

The plan literally decides what GEO measures. Content is briefed from the same plan that sets the prompt set we track. Until that link exists we are shipping a bundle; once it does, we are shipping a system.

`plan.decisions` is append-only. It is what makes the system explainable, and what the next planning cycle reads so it doesn't relitigate settled calls.

Coordination — events on the queue we already have

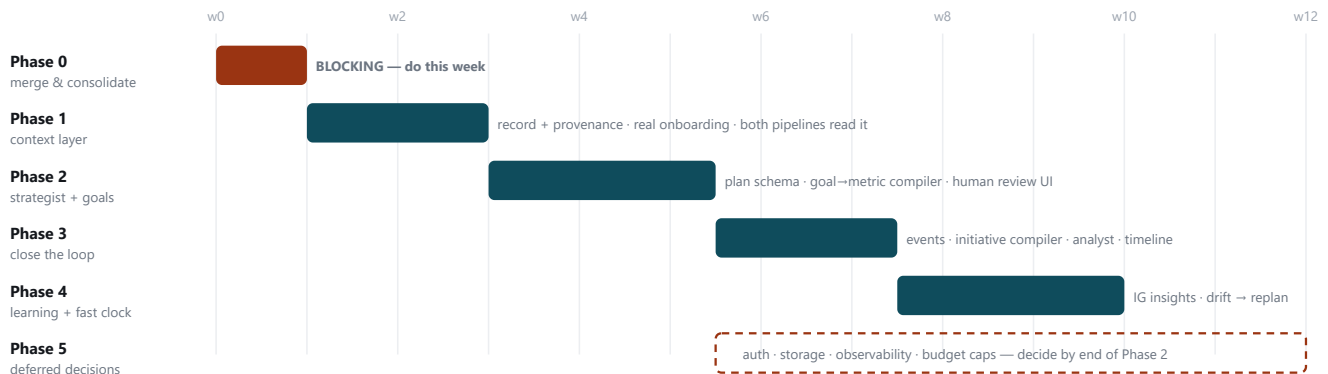
No Temporal, no LangGraph, no new orchestration dependency. An append-only domain event log plus a dispatcher mapping events to BullMQ jobs:

core.events	brand_id, type, payload, occurred_at, processed_at
brand.onboarded	→ strategist.plan
plan.published	→ initiatives.compile
initiative.created	→ content.generate geo.prompts.sync
campaign.completed	→ analyst.summarise
snapshot.computed	→ analyst.evaluate_goals
goal.at_risk	→ strategist.replan
context.drifted	→ strategist.replan

Loose coupling, zero new infrastructure, and a replayable history of everything the system did. BullMQ stays the transport; the event log is the record.

Phases

Sized for two engineers. Each phase ships something you can put in front of a room.



Twelve weeks, two engineers. Phase 0 is not optional and not parallelisable — every later phase assumes a single trunk.

PHASE 0 Merge and consolidate — blocking

~1 week

Nothing new. `feature/content-gen` and `geo/engine-mvp` both edit `packages/ai/`, `queues.ts`, `registry.ts` and `core.ts`; both independently created `packages/ai/src/adapters/gemini.llm.ts`. That conflict compounds every week we wait.

Resolve the duplicate Gemini adapter · land both branches on `main` · one `pnpm dev` that brings up both products · one seed script and one demo brand · green `typecheck && lint && build` on trunk.

Demo: both products running side by side on one database.

PHASE 1 Context layer

~2 weeks

`core.brand_context` with provenance · `assembleContext(brandId, task)` · onboarding as URL + handle → derived profile → six-question confirmation · refactor *both* pipelines to read context instead of ad-hoc brand fields · backfill.

Demo: onboard a real business in under three minutes; show one context object driving both a caption and a GEO analysis. Both products get better and we have added no agents.

PHASE 2 Strategist and goals

~2–3 weeks

`plan` schema · goal→metric compiler with hard validation · Strategist as a genuine tool-loop agent on the orchestrator role, with tools `read_context`, `read_geo_snapshots`, `read_competitors`, `read_performance`, `propose_plan` · plan review UI, human approves before anything executes · decision logging.

Demo: onboard → a real plan with measurable goals, pillars and a rationale you can argue with.

PHASE 3 Close the loop

~2 weeks

`core.events` + dispatcher · initiative compiler: content initiatives → generation jobs, **GEO initiatives** → **tracked prompts derived from the plan's themes** · Analyst computing goal progress from snapshots and post metrics, narrated · timeline view showing the GEO series with campaign markers — correlation shown, causation not claimed.

Demo: the full loop, end to end, on one screen.

PHASE 4 Learning and the fast clock

~2-3 weeks

Instagram insights ingestion → fast-loop metrics · performance facts written back into context as `source: 'social'` · drift detection → `goal.at_risk` → automatic replan proposal, still human-approved · Researcher agent: *why do answers cite these sources and not us?*

PHASE 5 The deferred decisions

decide by
w6

Auth, object storage, observability, payments, per-brand budget caps. Listed last, but **auth and storage become blocking during Phase 3** if anyone outside the team is to touch it. Decide them by the end of Phase 2.

Risks & open decisions

RISK	MITIGATION
Merge debt compounds; Phase 1 starts on two divergent trunks	Phase 0 is blocking. Do it this week.
We claim campaigns caused GEO movement	Two clocks; markers not attribution; state the limitation in the UI.
"Agent" everywhere → unpredictable cost and latency	Two agents only; everything else a workflow. Enforce in review.
Strategist emits pretty, unmeasurable plans	Goal→metric compiler rejects non-compiling goals in code.
Context layer becomes a six-week infrastructure project	One table, one function. No retrieval service before Phase 4.
Two people, three workstreams, twelve weeks	Orchestration needs a named owner before Phase 2 or it stalls.
Provider spend scales with brands × prompts × engines	<code>core.cost_events</code> exists; add per-brand caps before any pilot.
Answer-engine terms and pricing shift under us	The adapter boundary already isolates this; re-verify at each phase gate.

Open decisions — these need a human, not a design

#	QUESTION	RECOMMENDATION
1	Who owns <code>plan</code> / orchestration?	Settle first. It touches both products; unowned, it stalls.
2	How autonomous is re-planning?	Propose, never auto-apply, until we have eyeballed ~20 plans.
3	Is GEO a feature or a standalone product?	It is saleable alone — the answer changes packaging and possibly phase order.
4	Human approval before publishing?	Yes, until brand-critic rejection rates are known.
5	Auth, storage, observability choices	Needed by Phase 3. Decide by end of Phase 2.

Where to start

Phase 0, then `core.brand_context`. The context record is the highest-leverage object in the system: it makes both existing products better on its own, and it is the thing every later agent reads and writes. **Build it before any new agent.**